Procesamiento de imágenes usando geometría vectorial

IS-623 Computación gráfica

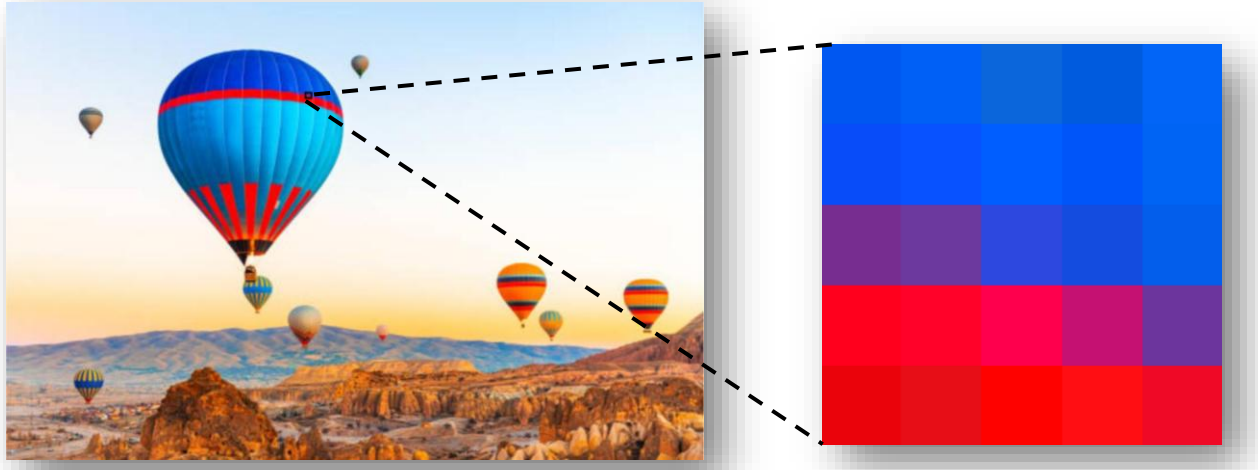
Docente: Juan Pablo Colina Morales

Imagen digital

Una **imagen digital** está compuesta por **píxeles**, que son las unidades mínimas de información. Cada píxel almacena un **valor numérico** que representa su color y nivel de intensidad (brillo).

La **cantidad y disposición de píxeles** determinan la resolución, es decir, el nivel de detalle y la cantidad de información que contiene la imagen.

Representación matricial de una imagen digital



R:0 G:87 B:241	R:0 G:96 B:246	R:11 G:102 B:219	R:0 G:91 B:222	R:2 G:101 B:247
R:8 G:76 B:249	R:8 G:82 B:255	R:0 G:94 B:255	R:0 G:85 B:249	R:0 G:100 B:244
R:119 G:45 B:144	R:108 G:57 B:158	R:45 G:72 B:223	R:20 G:76 B:223	R:3 G:94 B:235
R:255 G:0 B:29	R:255 G:2 B:41	R:254 G:0 B:77	R:197 G:17 B:114	R:108 G:53 B:157
R:234 G:3 B:11	R:231 G:14 B:23	R:254 G:2 B:0	R:255 G:14 B:19	R:239 G:9 B:38

Modelos de color

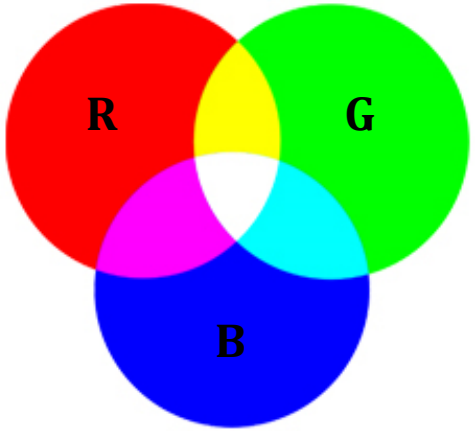
Modelo RGB

Es un modelo de color aditivo que genera colores combinando **rojo**, **verde** y **azul**.

Cada componente se representa con valores de 0 a 255, donde:

0 → sin intensidad
255 → intensidad máxima

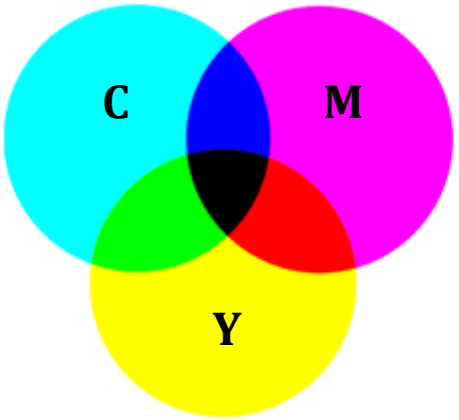
Es el sistema utilizado en pantallas e imágenes digitales.



Modelo CMYK

Es un modelo de color sustractivo utilizado en impresión, basado en la combinación de **cian**, **magenta**, **amarillo** y **negro**.

Se utiliza principalmente en impresoras y procesos de impresión física.

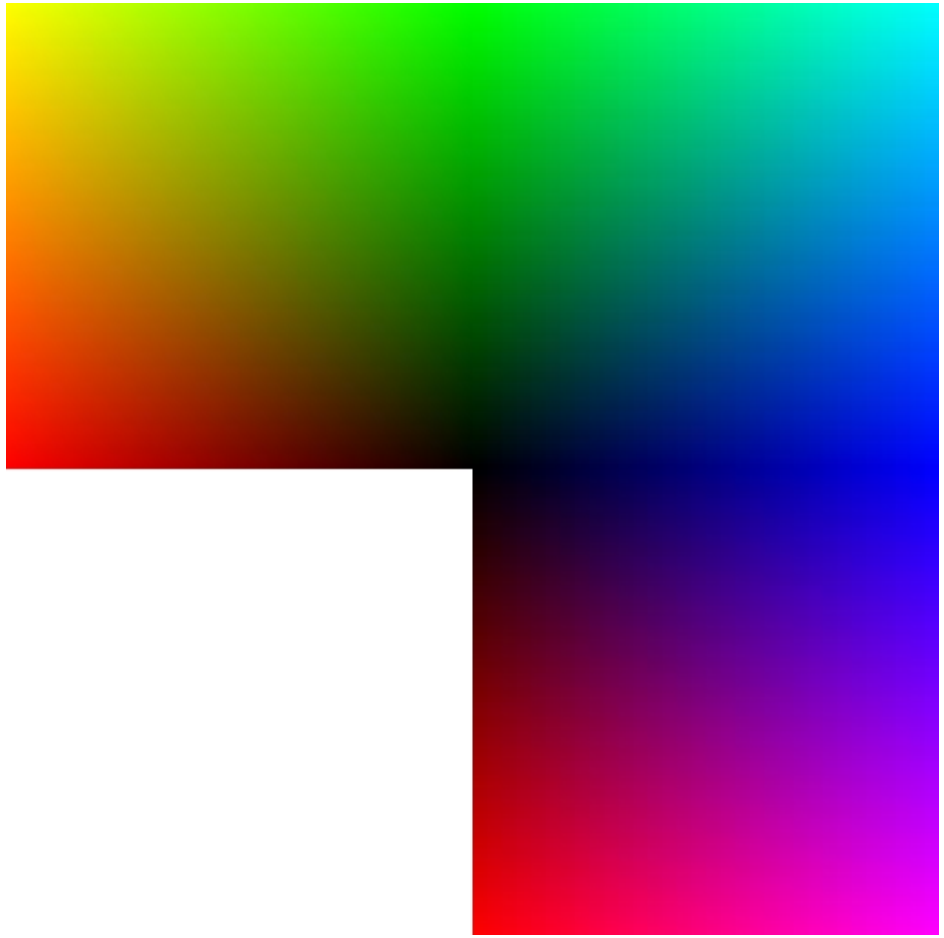


Mapa de Colores

Amarillo
(255, 255, 0)

Verde
(0, 255, 0)

Cian
(0, 255, 255)



Rojo
(255, 0, 0)

Azul
(0, 0, 255)

Magenta
(255, 0, 255)

Lectura y visualización de imágenes con Python

```python
import numpy as np
import matplotlib.pyplot as plt

ruta = "aurora.jpg"       # Ruta del archivo de imagen
img = plt.imread(ruta)    # Leer la imagen como matriz de píxeles

plt.imshow(img)           # Mostrar la imagen
plt.axis("off")           # Ocultar ejes
plt.show()                # Renderizar en pantalla
```

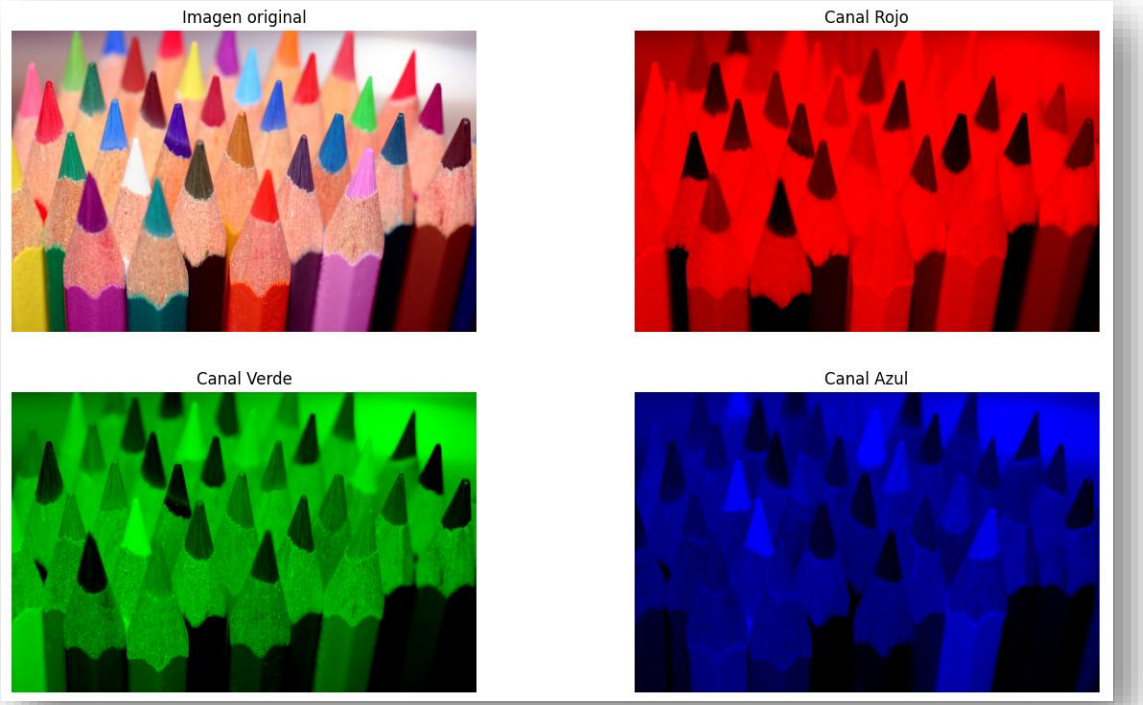
Tipo de dato y rango dinámico en imágenes digitales

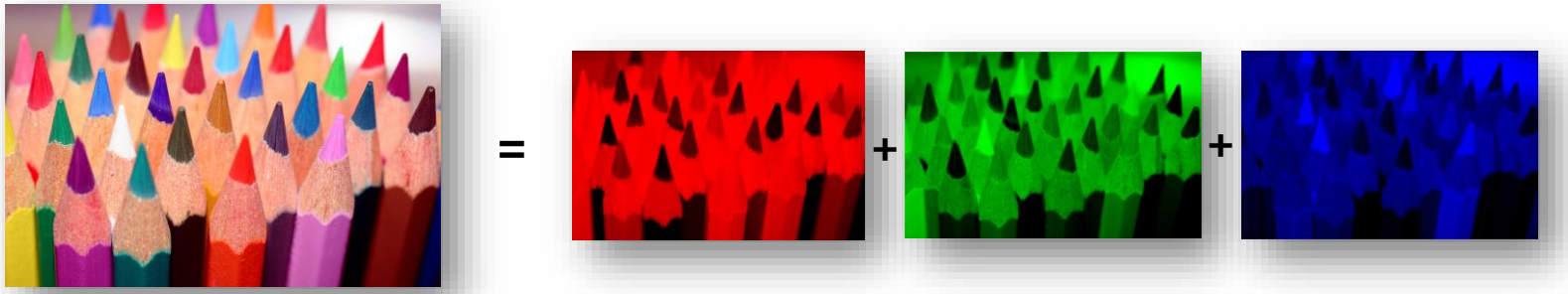
```python
import numpy as np
import matplotlib.pyplot as plt

ruta_jpg = "aurora.jpg"              # Ruta del archivo .jpg
img_jpg = plt.imread(ruta_jpg)       # Cargar imagen como matriz

ruta_png = "globos.png"              # Ruta del archivo .png
img_png = plt.imread(ruta_png)       # Cargar imagen como matriz

print("-------- IMAGEN JPG --------")
print("Tipo de dato:", img_jpg.dtype)     # Tipo de los píxeles
print("Tamaño:", img_jpg.shape)           # Dimensiones (alto, ancho, canales)

print("Valor mínimo:", np.min(img_jpg))   # Intensidad mínima
print("Valor máximo:", np.max(img_jpg))   # Intensidad máxima

print("-------- IMAGEN PNG --------")
print("Tipo de dato:", img_png.dtype)     # Tipo de los píxeles
print("Tamaño:", img_png.shape)           # Dimensiones (alto, ancho, canales)

print("Valor mínimo:", np.min(img_png))   # Intensidad mínima
print("Valor máximo:", np.max(img_png))   # Intensidad máxima
```

```
-------- IMAGEN JPG --------
Tipo de dato: uint8
Tamaño: (960, 1440, 3)
Valor mínimo: 0
Valor máximo: 255
```

```
-------- IMAGEN PNG --------
Tipo de dato: float32
Tamaño: (898, 1536, 4)
Valor mínimo: 0.0
Valor máximo: 1.0
```

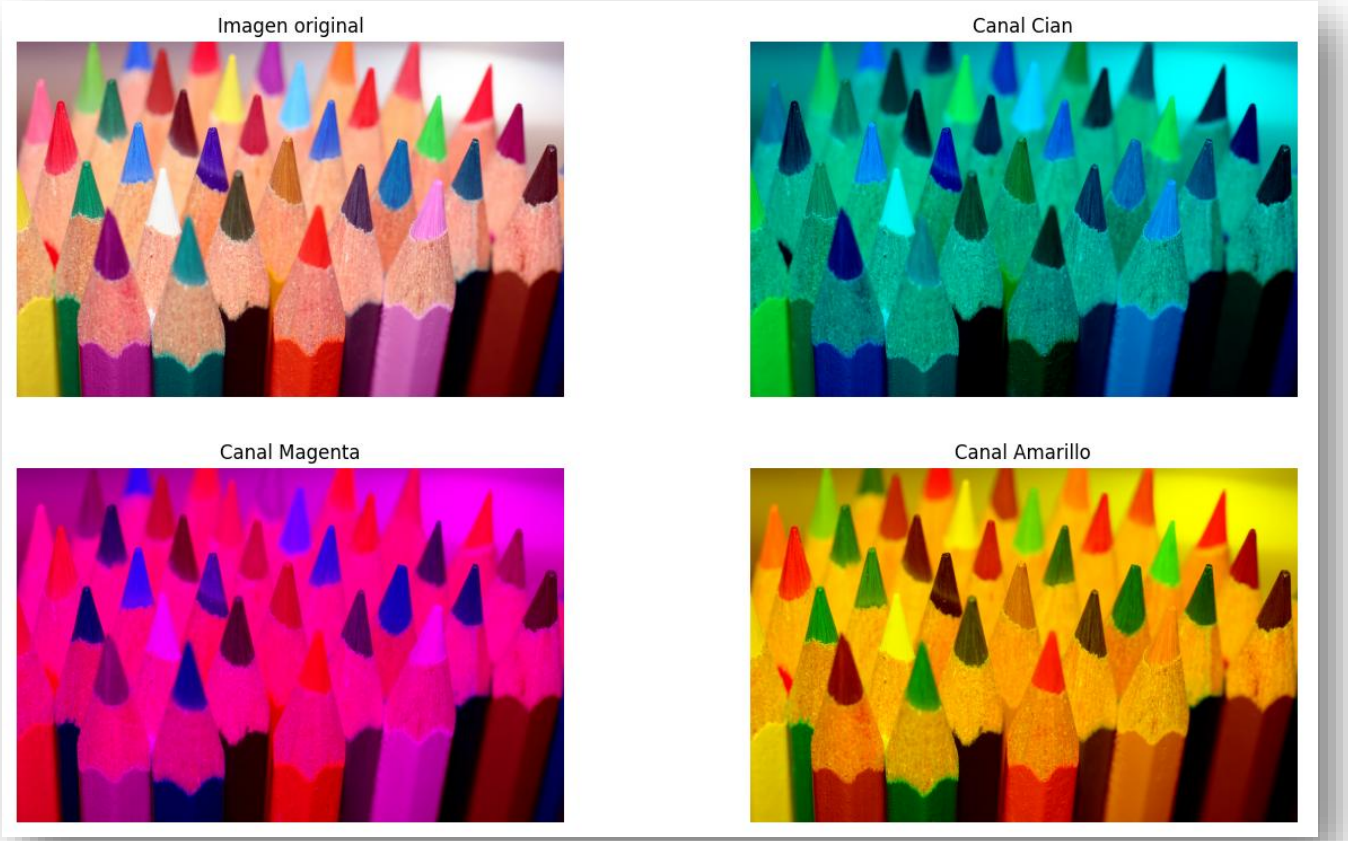
Descomposición RGB






```python
import numpy as np
import matplotlib.pyplot as plt

ruta  = "color.jpg"
img = plt.imread(ruta)

capaR = np.copy(img)
capaR[:,:,1] = capaR[:,:,2] = 0   # Canal rojo

capaG = np.copy(img)
capaG[:,:,0] = capaG[:,:,2] = 0   # Canal verde

capaB = np.copy(img)
capaB[:,:,0] = capaB[:,:,1] = 0   # Canal azul

plt.subplot(2,2,1)
plt.imshow(img)
plt.axis("off")
plt.title("Imagen original")

plt.subplot(2,2,2)
plt.imshow(capaR)
plt.axis("off")
plt.title("Canal Rojo")

plt.subplot(2,2,3)
plt.imshow(capaG)
plt.axis("off")
plt.title("Canal Verde")

plt.subplot(2,2,4)
plt.imshow(capaB)
plt.axis("off")
plt.title("Canal Azul")

plt.show()
```

Descomposición CMY




```python
import numpy as np
import matplotlib.pyplot as plt

ruta  = "color.jpg"
img = plt.imread(ruta)

capaC = np.copy(img)
capaC[:,:,0] = 0       # Canal cian

capaM = np.copy(img)
capaM[:,:,1] = 0       # Canal magenta

capaY = np.copy(img)
capaY[:,:,2] = 0       # Canal amarillo

plt.subplot(2,2,1)
plt.imshow(img)
plt.axis("off")
plt.title("Imagen original")

plt.subplot(2,2,2)
plt.imshow(capaC)
plt.axis("off")
plt.title("Canal Cian")

plt.subplot(2,2,3)
plt.imshow(capaM)
plt.axis("off")
plt.title("Canal Magenta")

plt.subplot(2,2,4)
plt.imshow(capaY)
plt.axis("off")
plt.title("Canal Amarillo")
plt.show()
```

Inversión de color

La inversión de color es una transformación digital que reemplaza cada valor de color por su valor complementario.

En imágenes RGB de 8 bits, se calcula como:

$$Nuevo_valor = 255 - Valor_original$$

Esto produce el efecto de “negativo” de la imagen, invirtiendo luces por sombras y colores por sus complementarios.

```python
import numpy as np
import matplotlib.pyplot as plt

ruta  = "color.jpg"
img = plt.imread(ruta)

plt.subplot(1,2,1)
plt.imshow(img)
plt.title("Imagen Original")
plt.axis("off")

inv = 255 - img

plt.subplot(1,2,2)
plt.imshow(inv)
plt.title("Inversa")
plt.axis("off")
plt.show()
```

Suma de imágenes

A

B

C

$$\underset{A}{\begin{bmatrix} 8 & 7 & 4 \\ 98 & 12 & 67 \\ 100 & 23 & 11 \end{bmatrix}} + \underset{B}{\begin{bmatrix} 34 & 77 & 99 \\ 100 & 23 & 55 \\ 33 & 88 & 11 \end{bmatrix}} = \underset{C}{\begin{bmatrix} 42 & 84 & 103 \\ 198 & 35 & 122 \\ 133 & 111 & 22 \end{bmatrix}}$$

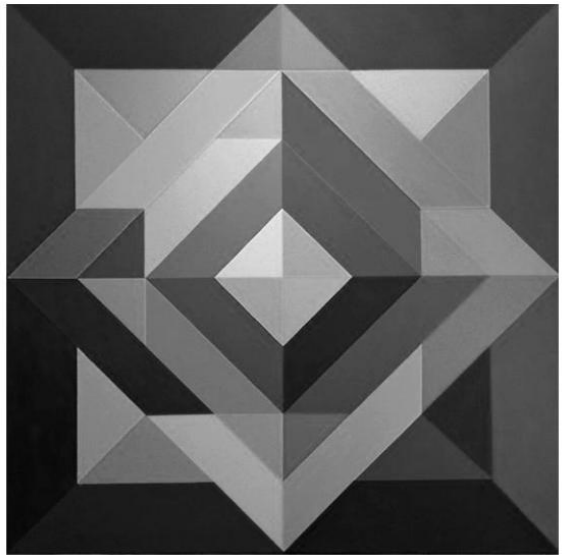
```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 1) Cargar ambas imágenes JPG
img_fondo = plt.imread("fondo.jpg")
img_subm = plt.imread("submarino.jpg")

print("dtype fondo:", img_fondo.dtype)
print("dtype submarino:", img_subm.dtype)

# 2) Igualar tamaño
h, w, _ = img_fondo.shape
img_subm = np.array(Image.fromarray(img_subm).resize((w, h)))

# 3) Convertir a float32 para evitar overflow
A = img_fondo.astype(np.float32)
B = img_subm.astype(np.float32)

# 4) Fusión por suma
fusion = (A + B)//2
fusion = np.clip(fusion, 0, 255).astype(np.uint8)

# 5) Mostrar
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img_fondo)
plt.title("Fondo JPG")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(img_subm)
plt.title("Submarino JPG")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(fusion)
plt.title("Suma de matrices")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Suma de imágenes ponderada

A B C

$$\alpha \cdot A + (1 - \alpha) \cdot B = C$$

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 1) Cargar ambas imágenes JPG
img_fondo = plt.imread("ciudad_dia.jpg")
img_subm = plt.imread("ciudad_noche.jpg")

# 2) Igualar tamaño
h, w, _ = img_fondo.shape
img_subm = np.array(Image.fromarray(img_subm).resize((w, h)))

# 3) Convertir a
A = img_fondo.astype(np.float32)
B = img_subm.astype(np.float32)

alpha = 0.3       # Constante alpha
# 4) Fusión ponderada
fusion = alpha*A + (1 - alpha)*B
fusion = np.clip(fusion, 0, 255).astype(np.uint8)


# 5) Mostrar
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img_fondo)
plt.title("Ciudad día")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(img_subm)
plt.title("Ciudad noche")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(fusion)
plt.title("Suma ponderada alpha = 0.3")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Escala de grises

La escala de grises es una representación de imagen donde cada píxel contiene un solo valor de intensidad, que indica qué tan claro u oscuro es.

No hay información de color, solo niveles de luminancia.

En imágenes de 8 bits:

0 → negro
255 → blanco
Valores intermedios → distintos tonos de gris

En el modelo RGB, una imagen en escala de grises ocurre cuando:

$$R = G = B$$

Es decir, los tres canales tienen la misma intensidad.

Técnica de promedio (Average)

```python
import numpy as np
import matplotlib.pyplot as plt

ruta = "cuadro_colores.jpg"

img = plt.imread(ruta)

img_gris_f =img.astype(np.float32)
img_gris = ((img_gris_f[:,:,0] + img_gris_f[:,:,1] + img_gris_f[:,:,2])/3).astype(np.uint8)

plt.imshow(img_gris, cmap="gray")
plt.axis("off")
plt.title("Gris promedio")
plt.show()
```

La técnica promedio para convertir una imagen a escala de grises consiste en calcular la media aritmética de los tres canales de color (R, G y B) en cada píxel:

$$I_{GRIS} = \frac{R + G + B}{3}$$

Todos los canales tienen el mismo peso, por lo que es un método simple y directo.

Técnica de luminosidad (Luminosity)

```python
import numpy as np
import matplotlib.pyplot as plt

img = plt.imread("cuadro_colores.jpg")

img = img.astype(np.float32)

# Técnica de luminosidad
ImgGris = 0.299*img[:,:,0] + 0.587*img[:,:,1] + 0.114*img[:,:,2]

# Convertir nuevamente a uint8
ImgGris = np.clip(ImgGris, 0, 255).astype(np.uint8)

# Mostrar resultado
plt.figure("Escala de grises (Luminosidad)")
plt.imshow(ImgGris, cmap="gray")
plt.axis("off")
plt.show()
```

La técnica de luminosidad es un método para convertir una imagen a escala de grises asignando pesos diferentes a los canales R, G y B, según la sensibilidad del ojo humano a cada color.

$$I_{GRIS} = 0.299R + 0.587G + 0.114B$$

El canal verde tiene mayor peso porque el sistema visual humano percibe con mayor intensidad esa componente.

Este método produce una representación más realista que el promedio simple.

Técnica de tonalidad (Midgray)

```python
import numpy as np
import matplotlib.pyplot as plt

img = plt.imread("cuadro_colores.jpg").astype(np.float32)

mx = np.max(img, axis=2)
mn = np.min(img, axis=2)
ImgGris = (mx + mn) / 2

ImgGris = np.clip(ImgGris, 0, 255).astype(np.uint8)

plt.imshow(ImgGris, cmap="gray")
plt.axis("off")
plt.show()
```

La técnica de tonalidad convierte una imagen a escala de grises tomando el promedio entre el valor máximo y el valor mínimo de los tres canales RGB en cada píxel.

$$I_{GRIS} = \frac{\max(R, G, B) + \min(R, G, B)}{2}$$

La idea es estimar un tono intermedio entre la mayor y menor intensidad del píxel, produciendo un gris basado en el rango dinámico del color.

Es un método simple que mantiene mejor el contraste que el promedio, pero no es perceptualmente tan preciso como la técnica de luminosidad.

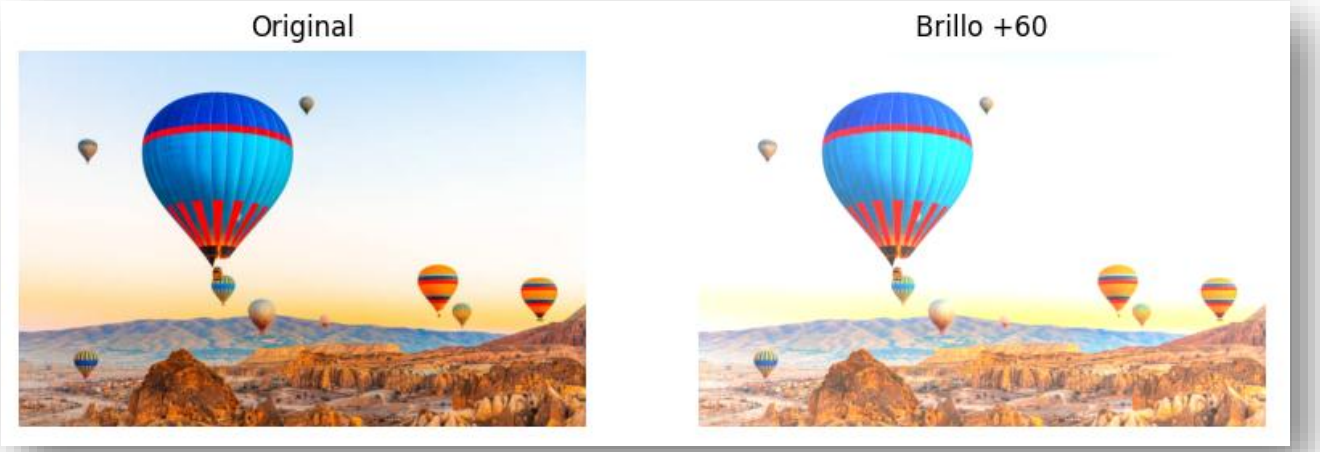
Ajuste de brillo

El ajuste de brillo modifica la luminancia de una imagen sumando una constante a cada píxel:Matemáticamente se expresa como:

$$I_{mod} = I + b$$

- Si $b > 0$ la imagen se aclara.
- Si $b < 0$ se oscurece.

En imágenes de 8 bits, los valores deben mantenerse en el rango 0–255, aplicando saturación para evitar desbordamientos.




```python
import numpy as np
import matplotlib.pyplot as plt

# 1) Abrir imagen JPG
ruta = "globo.jpg"
img = plt.imread(ruta)

# (1) Original
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.axis("off")
plt.title("Original")

# (2) Brillo +60
imgf = img.astype(np.float32)
img_clip = (np.clip(imgf + 60, 0, 255)).astype(np.uint8)

plt.subplot(1, 2, 2)
plt.imshow(img_clip)
plt.axis("off")
plt.title("Brillo +60")
plt.show()
```

Ajuste de canal

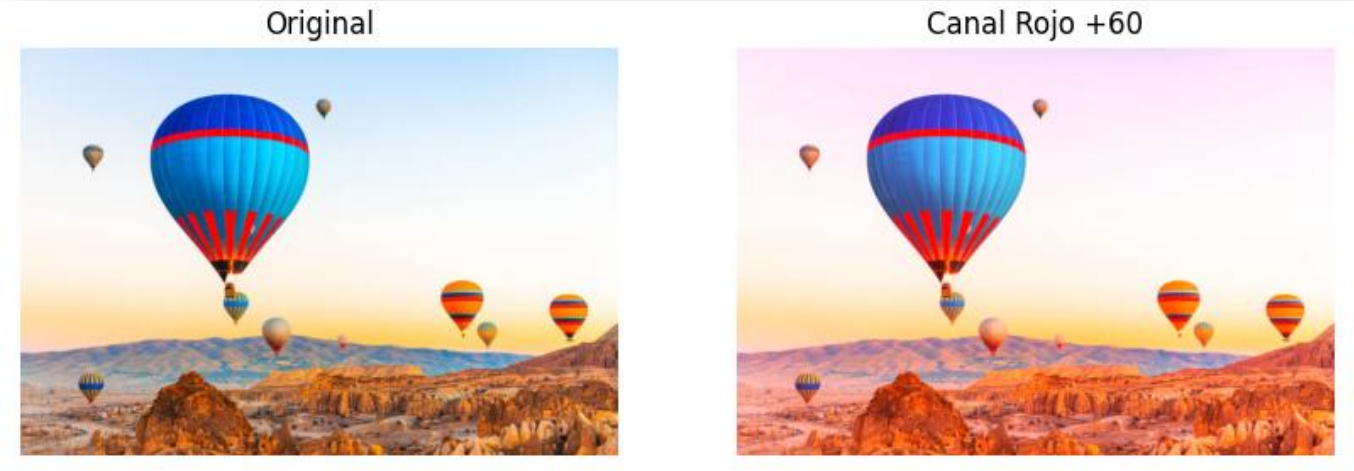
El ajuste de brillo por canal modifica de manera independiente la intensidad de un canal específico (R, G o B) en una imagen RGB:

$$I_{mod} = I(c) + b$$

donde c es el canal seleccionado y b el incremento.

A diferencia del brillo global, este ajuste también altera el balance de color, generando dominantes (más rojo → tonos cálidos, más azul → tonos fríos).

Es una técnica utilizada en corrección y calibración de color.




```python
import numpy as np
import matplotlib.pyplot as plt

ruta = "globo.jpg"
img = plt.imread(ruta)

plt.figure(figsize=(10,5))

# Imagen original
plt.subplot(1,2,1)
plt.imshow(img)
plt.axis("off")
plt.title("Original")

# Convertir a float para operar
img_R = img.astype(np.float32).copy()

# Alterar solo el canal rojo (R = canal 0)
img_R[:,:,0] = np.clip(img_R[:,:,0] + 60, 0, 255)

# Volver a uint8
img_R = img_R.astype(np.uint8)

plt.subplot(1,2,2)
plt.imshow(img_R)
plt.axis("off")
plt.title("Canal Rojo +60")

plt.show()
```

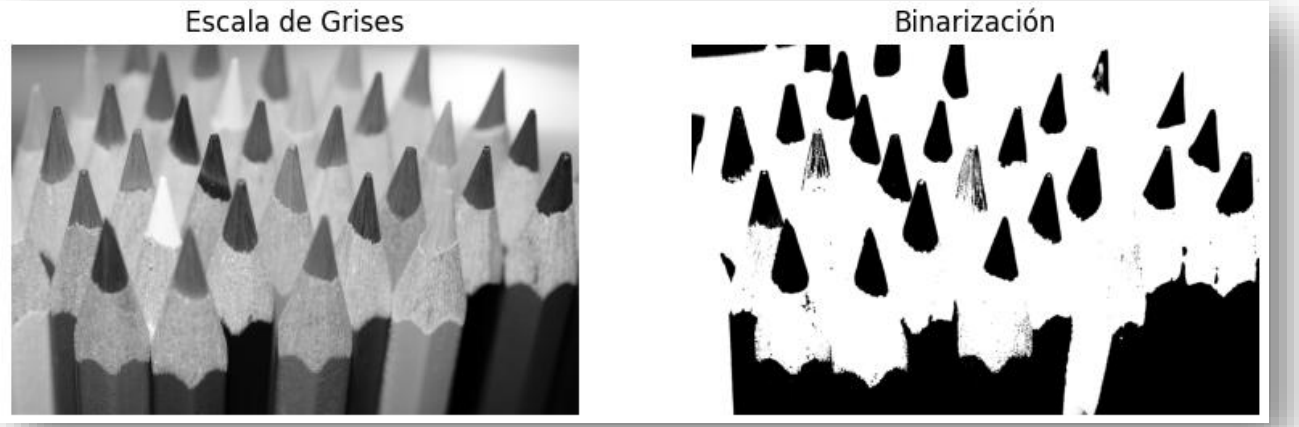
Binarización

La **binarización** convierte una imagen en escala de grises en una imagen de solo dos niveles: negro y blanco, usando un umbral T:

$$I_{bin}(i,j) = \begin{cases} 255, & I(i,j) \geq T \\ 0, & I(i,j) < T \end{cases}$$

donde $I(i,j)$ es la intensidad original y T es el valor de umbral.

Se utiliza en segmentación y reconocimiento, ya que simplifica la imagen eliminando los niveles intermedios de gris.




```python
import numpy as np
import matplotlib.pyplot as plt

# Cargar imagen RGB
img = plt.imread("color.jpg").astype(np.float32)

# Convertir a escala de grises
img_gray = 0.299*img[:,:,0] + 0.587*img[:,:,1] + 0.114*img[:,:,2]

# Definir umbral
T = 128

# Binarización
img_bin = np.where(img_gray >= T, 255, 0).astype(np.uint8)

# Mostrar resultados
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_gray, cmap="gray")
plt.title("Escala de Grises")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(img_bin, cmap="gray")
plt.title("Binarización")
plt.axis("off")

plt.show()
```

Traslación de una imagen

La traslación es una transformación geométrica afín que desplaza todos los píxeles de una imagen una distancia fija en el plano.

Si $(x\ , y\ )$ es la posición original, la nueva posición se calcula como:

$$(x', y') = \ (x\ +\ \Delta x, y\ +\ \Delta y)$$

donde:

- Δx es el desplazamiento horizontal (positivo → derecha, negativo → izquierda).
- Δy es el desplazamiento vertical (positivo → abajo, negativo → arriba).

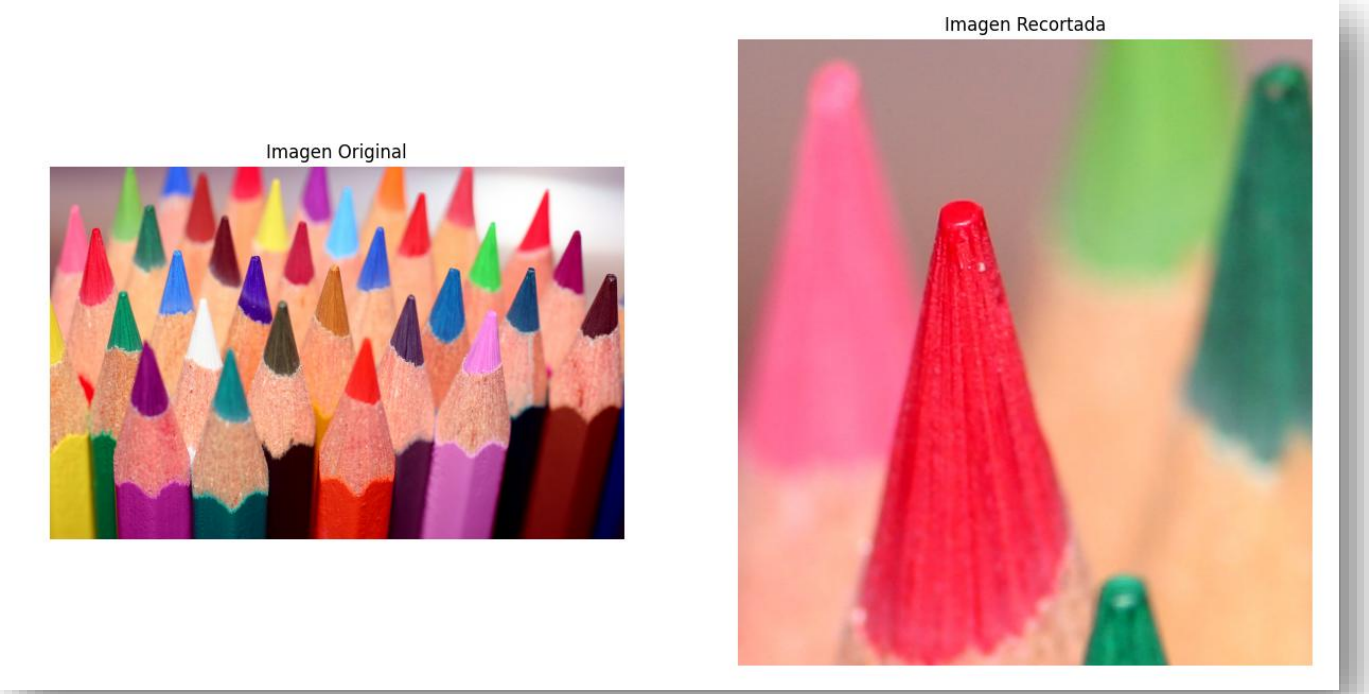
Esta operación mueve la imagen sin alterar su forma, tamaño ni orientación.

```python
import numpy as np
import matplotlib.pyplot as plt

img = plt.imread("color.jpg")
h, w = img.shape[:2]

dx = 150   # + derecha, - izquierda
dy = 150   # + abajo,  - arriba

trasladada = np.zeros_like(img)

# Coordenadas en la imagen destino
x1d = max(0, dx)
y1d = max(0, dy)
x2d = min(w, w + dx)
y2d = min(h, h + dy)

# Coordenadas correspondientes en la imagen origen
x1s = max(0, -dx)
y1s = max(0, -dy)
x2s = x1s + (x2d - x1d)
y2s = y1s + (y2d - y1d)

# Copia solo si hay intersección válida
if x2d > x1d and y2d > y1d:
    trasladada[y1d:y2d, x1d:x2d] = img[y1s:y2s, x1s:x2s]

plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(img)
plt.axis("off")

plt.subplot(1,2,2)
plt.title(f"Trasladada dx={dx}, dy={dy}")
plt.imshow(trasladada)
plt.axis("off")
plt.show()
```

Recorte de imagen

```python
import numpy as np
import matplotlib.pyplot as plt

img1 = plt.imread("color.jpg")

xIni, xFin = 50, 600   # columnas
yIni, yFin = 200, 800   # filas

ImgRecorte = img1[yIni:yFin, xIni:xFin]

plt.subplot(1,2,1)
plt.title("Imagen Original")
plt.imshow(img1)
plt.axis("off")

plt.subplot(1,2,2)
plt.title("Imagen Recortada")
plt.imshow(ImgRecorte)
plt.axis("off")

plt.show()
```

El recorte de una imagen es una operación que extrae una región específica definida por un rango de coordenadas.



Rotar una imagen

La rotación de una imagen es una transformación geométrica que gira todos los píxeles un ángulo θ alrededor de un punto (generalmente el centro).

Las nuevas coordenadas se calculan mediante la matriz de rotación:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} cos\,\theta & -sin\,\theta \\ sin\,\theta & cos\,\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Esta operación cambia la orientación de la imagen sin alterar su contenido, aunque puede requerir interpolación y mayor costo computacional.

Original

Rotada 45°

```python
import numpy as np
import matplotlib.pyplot as plt


def rotar_manual(img, ang_deg, fondo=0):
    """
    Rotación MANUAL usando mapeo inverso + vecino más cercano.

    Parámetros
    ----------
    img : np.ndarray
        Imagen de entrada (H,W) en gris o (H,W,3) RGB.
    ang_deg : float
        Ángulo en grados (positivo = antihorario).
    fondo : int o float
        Valor de fondo para zonas vacías (0 = negro).

    Retorna
    -------
    out : np.ndarray
        Imagen rotada, con dimensiones ajustadas para contener toda la rotación.
    """

    # 1) Preparar datos
    ang = np.deg2rad(ang_deg)                # Convertir grados a radianes
    cos_t, sin_t = np.cos(ang), np.sin(ang)   # cos(θ), sin(θ)

    H, W = img.shape[:2]
    es_rgb = (img.ndim == 3)

    # Centro de la imagen original (coordenadas continuas)
    cx, cy = (W - 1) / 2.0, (H - 1) / 2.0

    # 2) Calcular el tamaño de la imagen de salida (bounding box)
    #    Rotamos las 4 esquinas alrededor del centro y buscamos min/max.
    esquinas = np.array([
        [0,   0],
        [W-1, 0],
        [W-1, H-1],
        [0,   H-1]
    ], dtype=np.float32)
```

```python
    # Trasladar esquinas al sistema centrado (0,0 en el centro)
    x = esquinas[:, 0] - cx
    y = esquinas[:, 1] - cy

    # Aplicar rotación a las esquinas (en el sistema centrado)
    xr = x * cos_t - y * sin_t
    yr = x * sin_t + y * cos_t

    # Extremos en el sistema rotado
    min_x, max_x = xr.min(), xr.max()
    min_y, max_y = yr.min(), yr.max()

    # Dimensiones necesarias para contener todo
    out_W = int(np.ceil(max_x - min_x + 1))
    out_H = int(np.ceil(max_y - min_y + 1))

    # Centro de la imagen de salida
    out_cx, out_cy = (out_W - 1) / 2.0, (out_H - 1) / 2.0

    # 3) Crear imagen de salida (rellena con fondo)
    if es_rgb:
        out = np.full((out_H, out_W, 3), fondo, dtype=img.dtype)
    else:
        out = np.full((out_H, out_W), fondo, dtype=img.dtype)

    # 4) Mapeo inverso:
    #    Para cada píxel destino (x',y') calculo el (x,y) en la imagen original.
    #    Inversa de la rotación:
    #       [x] = [ cosθ  sinθ][x']
    #       [y]   [-sinθ cosθ][y']
    #
    #    Ojo: todo en sistema centrado.
    for y_out in range(out_H):
        for x_out in range(out_W):

            # Coordenadas del píxel destino en sistema centrado
            x2 = x_out - out_cx
            y2 = y_out - out_cy

            # Aplicar rotación inversa (desrotar) para ir a la imagen original
            x1 = x2 * cos_t + y2 * sin_t
            y1 = -x2 * sin_t + y2 * cos_t
```

```python
            # Volver al sistema original (descentrar)
            x_src = x1 + cx
            y_src = y1 + cy

            # 5) Vecino más cercano: redondear a entero
            xi = int(round(x_src))
            yi = int(round(y_src))

            # 6) Si cae dentro de la imagen original, copiar el píxel
            if 0 <= xi < W and 0 <= yi < H:
                out[y_out, x_out] = img[yi, xi]

    return out



img = plt.imread("color.jpg")

# Para trabajar en gris:
# img = img.astype(np.float32)
# img = 0.299*img[:,:,0] + 0.587*img[:,:,1] + 0.114*img[:,:,2]
ang_deg=45
rotada = rotar_manual(img, ang_deg, fondo=0)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(img, cmap="gray" if img.ndim==2 else None)
plt.axis("off")

plt.subplot(1,2,2)
plt.title(f"Rotada {ang_deg}°")
plt.imshow(rotada, cmap="gray" if rotada.ndim==2 else None)
plt.axis("off")
plt.show()
```

Reducción de resolución

La reducción de resolución es el proceso de disminuir el número de píxeles de una imagen, lo que implica una pérdida de detalle visual.

En el submuestreo directo, la imagen reducida se obtiene tomando un píxel cada f posiciones en ambas direcciones:

$$I_{reducida}(i,j) = I(i \cdot f, j \cdot f)$$

donde f es el factor de reducción.

Este método conserva solo muestras periódicas de la imagen original, sin interpolación.

$$I = \begin{bmatrix} A & B & C & D & E & F \\ G & H & I & J & K & L \\ M & N & O & P & Q & R \\ S & T & U & V & W & X \\ Y & Z & a & b & c & d \\ e & f & g & h & i & j \end{bmatrix}$$

Reducción con factor de $f = 2$

$$I_{\text{reducida}} = \begin{bmatrix} A & C & E \\ M & O & Q \\ Y & a & c \end{bmatrix}$$

```python
import numpy as np
import matplotlib.pyplot as plt

# Cargar imagen
img = plt.imread("color.jpg")

# Factor de reducción
f = 30

# Submuestreo directo (downsampling)
img_reducida = img[::f, ::f]

# Mostrar tamaños
print("Tamaño original:", img.shape)
print("Tamaño reducido:", img_reducida.shape)

# Visualización
plt.figure(figsize=(8,6))

plt.subplot(2,1,1)
plt.title("Imagen Original")
plt.imshow(img)
plt.axis("off")

plt.subplot(2,1,2)
plt.title(f"Imagen Reducida (f = {f})")
plt.imshow(img_reducida)
plt.axis("off")

plt.show()
```

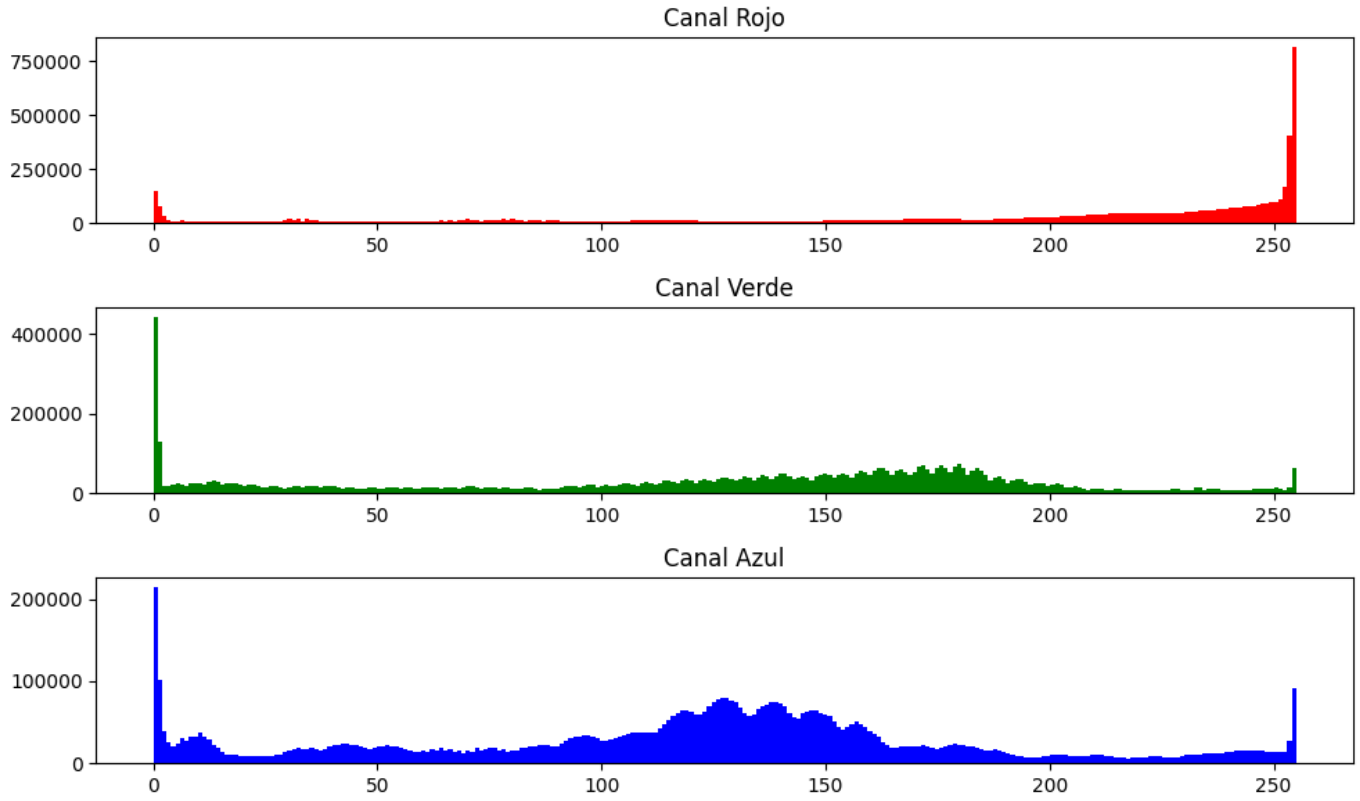
Ampliación de una zona

```python
import numpy as np
import matplotlib.pyplot as plt

# Leer imagen (sin normalizar)
img1 = plt.imread('color.jpg')

# Obtener alto y ancho
h, w = img1.shape[:2]

# Tamaño del recorte central
zoom_area = 500

# Coordenadas del centro
start_row = h // 2 - zoom_area // 2
end_row   = h // 2 + zoom_area // 2
start_col = w // 2 - zoom_area // 2
end_col   = w // 2 + zoom_area // 2

# Recortar región central
recorte = img1[start_row:end_row, start_col:end_col]

# Factor de ampliación
zoom_factor = 5

# Zoom repitiendo píxeles
zoomed = np.kron(recorte, np.ones((zoom_factor, zoom_factor, 1)))

# Convertir al mismo tipo de la imagen original
zoomed = zoomed.astype(img1.dtype)

# Mostrar resultados
plt.figure("Zoom hacia adentro")

plt.subplot(1, 2, 1)
plt.title("Imagen Original")
plt.imshow(img1)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Zoom sobre región central")
plt.imshow(zoomed)
plt.axis('off')

plt.show()
```

Histograma de una imagen

Un histograma es una representación gráfica que muestra cómo se distribuyen los valores de intensidad o datos dentro de un conjunto.

En imágenes digitales, un histograma indica cuántos píxeles existen para cada nivel de intensidad, normalmente en el rango 0 a 255, permitiendo analizar el brillo, contraste y distribución tonal de la imagen.

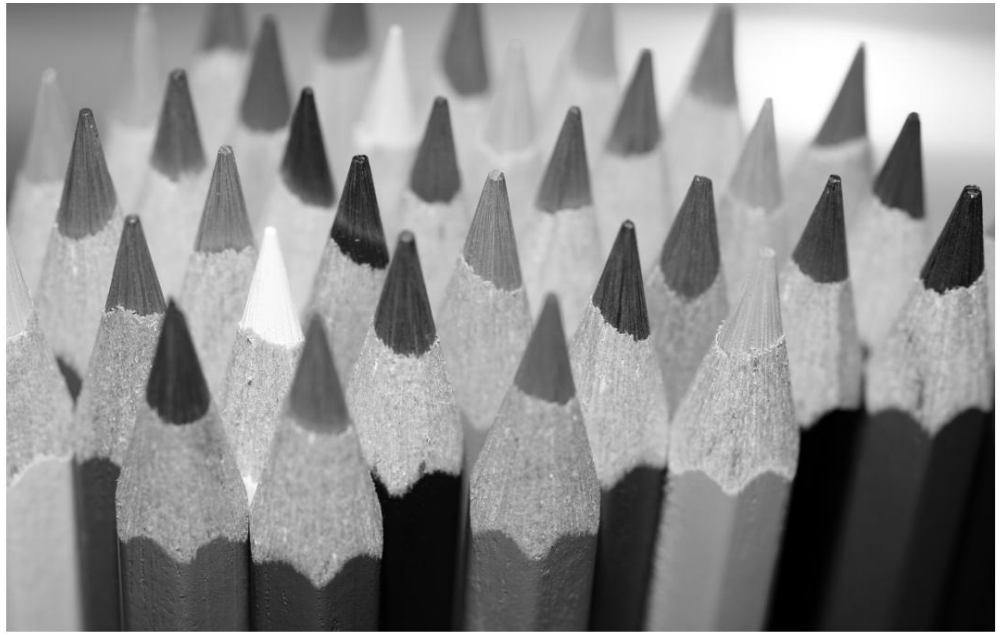


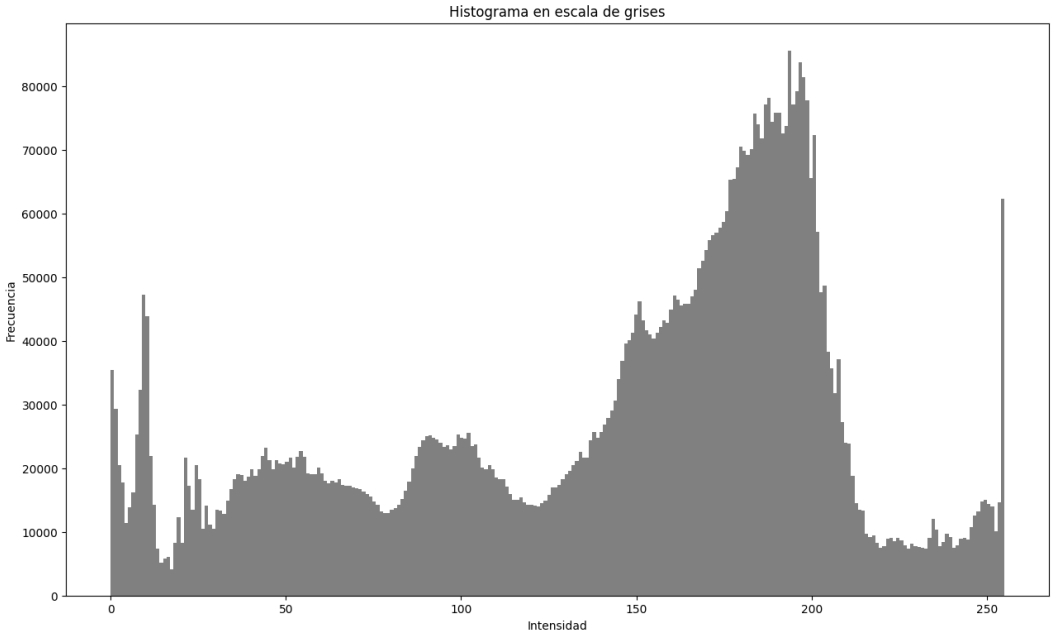
¿Qué representa el histograma?

- Eje X: niveles de intensidad de la imagen (0 a 255).
- Eje Y: cantidad de píxeles que poseen cada nivel de intensidad.

¿Para qué se utiliza?

- **Analizar el brillo:** si los valores se concentran hacia la izquierda, la imagen es oscura; si se agrupan hacia la derecha, la imagen es clara.
- **Evaluar el contraste:** cuando los valores están en un rango reducido, el contraste es bajo; si se distribuyen ampliamente, el contraste es mayor.
- **Mejorar la imagen:** permite ajustar brillo, contraste y exposición, además de aplicar técnicas como la ecualización del histograma.





```python
# Importar librerías
import numpy as np
import matplotlib.pyplot as plt

# Leer imagen
img = plt.imread('color.jpg')

# Convertir a rango 0-255 si está en 0-1
if img.max() <= 1.0:
    img = (img * 255).astype(np.uint8)

# Separar canales RGB
R = img[:, :, 0]
G = img[:, :, 1]
B = img[:, :, 2]

# Crear figura
plt.figure(figsize=(10, 6))

# Histograma rojo
plt.subplot(3, 1, 1)
plt.hist(R.ravel(), bins=256, color='red')
plt.title('Canal Rojo')

# Histograma verde
plt.subplot(3, 1, 2)
plt.hist(G.ravel(), bins=256, color='green')
plt.title('Canal Verde')

# Histograma azul
plt.subplot(3, 1, 3)
plt.hist(B.ravel(), bins=256, color='blue')
plt.title('Canal Azul')

plt.tight_layout()
plt.show()
```

Detectores de bordes

¿Qué es un borde?

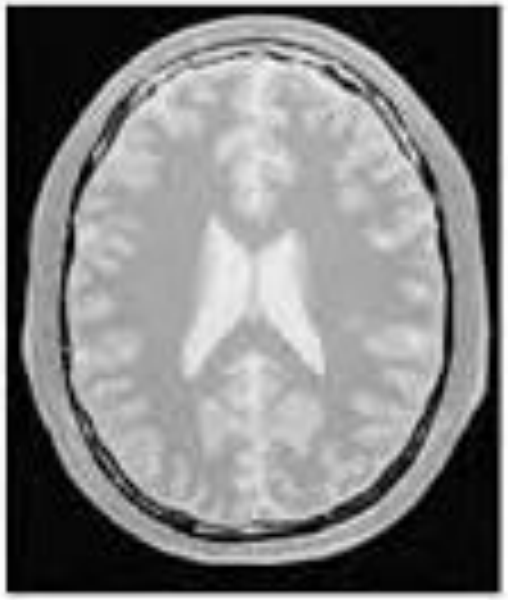
Un borde es una zona de la imagen donde ocurre un cambio brusco de intensidad.

Matemáticamente:

Borde ⇒ *gran variación de intensidad*

Ejemplo:

- cambio de negro a blanco
- límite entre dos objetos
- contornos



Idea fundamental de los detectores de bordes

Los bordes aparecen donde la imagen cambia rápidamente.

Eso se mide usando la **derivada**.

Si la imagen es:

$$I(x, y)$$

entonces los bordes se detectan calculando el **gradiente**:

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right)$$

Magnitud del borde:

$$| \nabla I | = \sqrt{G_x^2 + G_y^2}$$

¿Qué produce un detector de bordes?

Un detector de bordes analiza los cambios de intensidad en una imagen para encontrar los contornos de los objetos.

- El resultado no es una línea directamente, sino una nueva imagen llamada:

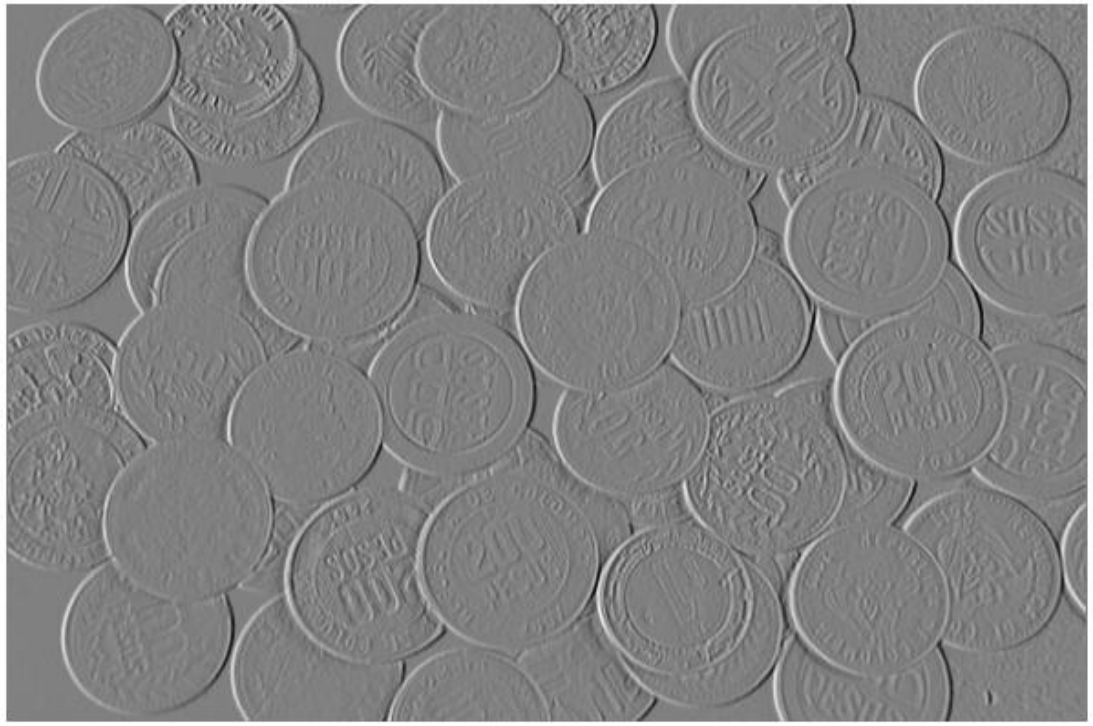
<u>Mapa de bordes</u>

En esta imagen:

- Cada píxel almacena qué tan fuerte es el cambio de intensidad.

Interpretación

- Valores bajos → zona uniforme
- Valores altos → posible borde



Diferentes filtros detectores de bordes

- Filtro Roberts
- Filtro Prewitt
- Filtro Sobel
- Laplaciano
- Canny